

Fine-Tuning LLMs and RLHF

Naeemullah Khan

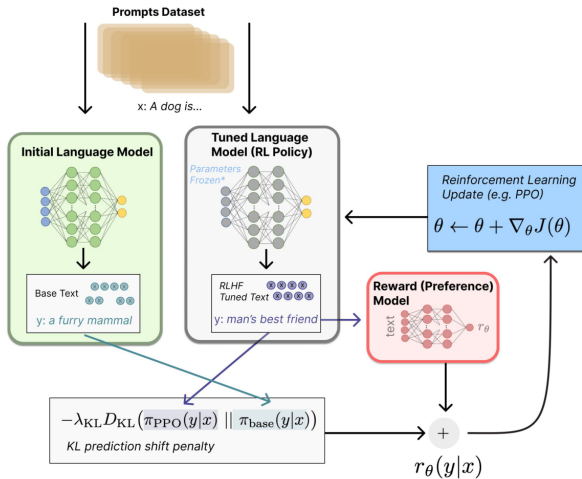
naeemullah.khan@kaust.edu.sa



جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

August 9, 2026



Lambert, Castricato, von Werra & Havrilla, "Illustrating Reinforcement Learning from Human Feedback (RLHF)," Hugging Face Blog, 2022.

1. Motivation
2. Learning Outcomes
3. Fine-Tuning Methods for LLMs
 - 3.1 Categories of Fine-Tuning
 - 3.2 Full Fine-Tuning (FT)
 - 3.3 Parameter-Efficient Fine-Tuning (PEFT)
4. Supervised Fine-Tuning (SFT)
 - 4.1 What is SFT?
 - 4.2 Datasets for SFT
 - 4.3 Domain-Specific SFT
 - 4.4 Limitations of SFT
5. RLHF — Reinforcement Learning with Human Feedback
 - 5.1 Why RLHF?
 - 5.2 The RLHF Pipeline

- 6. Reward Model (RM)
- 7. RLHF Training with PPO
- 8. Direct Preference Optimization (DPO)
- 9. LoRA and Quantized LoRA
 - 9.1 What is LoRA?
 - 9.2 Benefits of LoRA
 - 9.3 What is QLoRA?
 - 9.4 Applications of LoRA / QLoRA
- 10. Limitations of Fine-Tuning and RLHF
- 11. Future Directions
- 12. Summary

- ▶ Pre-trained LLMs are powerful but not aligned to specific use-cases or human preferences.

Motivating Examples:

- ▷ GPT-3 → InstructGPT
- ▷ LLaMA → Alpaca, Vicuna
- ▷ ChatGPT & Claude → RLHF-tuned

- ▶ Pre-trained LLMs are powerful but not aligned to specific use-cases or human preferences.
- ▶ **We need:**
 - Adaptability (domain tuning)
 - Alignment (user intention)
 - Efficiency (cost-effective methods)

Motivating Examples:

- ▷ GPT-3 → InstructGPT
- ▷ LLaMA → Alpaca, Vicuna
- ▷ ChatGPT & Claude → RLHF-tuned

- ▶ Pre-trained LLMs are powerful but not aligned to specific use-cases or human preferences.
- ▶ **We need:**
 - Adaptability (domain tuning)
 - Alignment (user intention)
 - Efficiency (cost-effective methods)
- ▶ Fine-tuning and RLHF bridge the gap between general intelligence and practical usability.

Motivating Examples:

- ▷ GPT-3 → InstructGPT
- ▷ LLaMA → Alpaca, Vicuna
- ▷ ChatGPT & Claude → RLHF-tuned

After this session, you will be able to:

- ▶ Explain and compare different fine-tuning methods for LLMs
- ▶ Understand the pipeline of Supervised Fine-Tuning and RLHF
- ▶ Describe PPO (Proximal Policy Optimization) and DPO (Direct Preference Optimization)
- ▶ Apply Low-Rank Adaptation (LoRA) and Quantized LoRA for efficient tuning
- ▶ Analyze trade-offs, limitations, and future directions of LLM adaptation

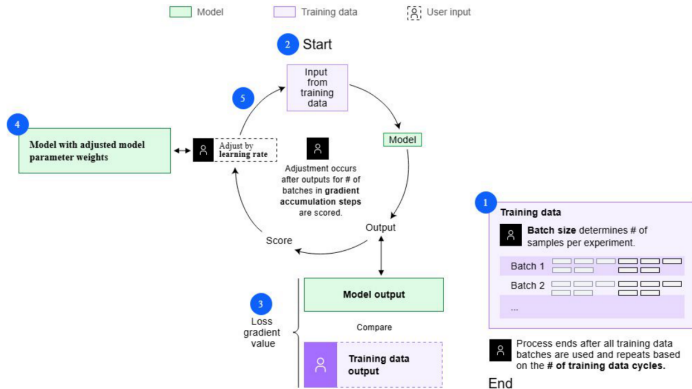
Fine-Tuning Methods for LLMs

Method	Description	Trained params	Cost
Full FT	Retrain all weights	All	Expensive
Adapter	Insert small bottleneck layers (Houlsby et al., 2019)	Few	Efficient
Prefix / Prompt	Tune continuous prompt vectors, base model frozen	Few tokens	Efficient
LoRA / QLoRA	Low-rank weight deltas	Very few	Very efficient
Instruction tuning	Tune on instruction-response datasets	All or part	Varies

Instruction tuning is an orthogonal axis: it describes *what data* the model is tuned on, and can be done with any of the methods above.

- ▶ **Definition:** Fine-tune all parameters of the pre-trained model on downstream data.
- ▶ **Historical Use:** Common in early GPT-2 and BERT applications.
- ▶ **Pros:**
 - Maximum flexibility and performance
- ▶ **Cons:**
 - Expensive (requires large compute resources)
 - Prone to catastrophic forgetting
 - Not efficient for large models

Full Fine-Tuning Workflow

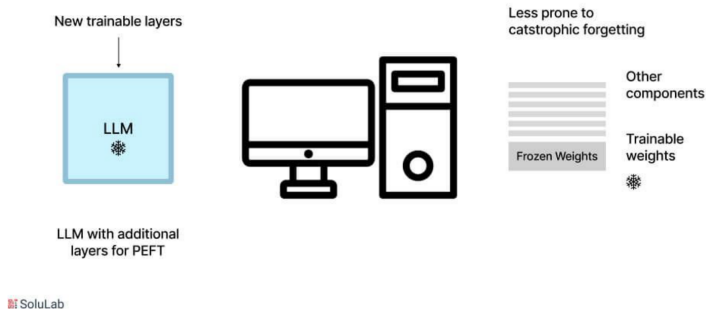


Source: IBM watsonx.ai documentation, "Full fine tuning,"

<https://www.ibm.com/docs/en/watsonx/w-and-w/2.1.0?topic=tuning-full-fine>.

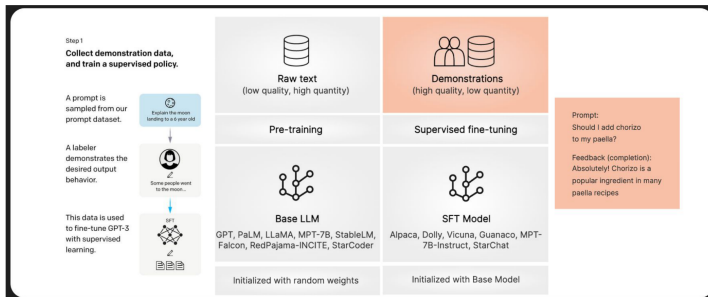
- ▶ **Key Idea:** Keep the base model frozen, tune only a small subset of parameters.
- ▶ **Types:**
 - Adapter Modules (Houlsby et al., 2019)
 - Prefix Tuning (Li & Liang, 2021) and Prompt Tuning (Lester et al., 2021)
 - LoRA / QLoRA
- ▶ **Benefits:**
 - Enables multi-tasking and personalization
 - Suitable for low-resource adaptation
 - Reduces compute and memory requirements

Parameter efficient fine-tuning (PEFT)



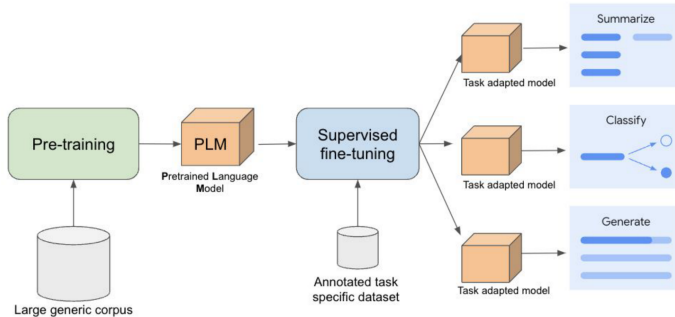
Supervised Fine-Tuning (SFT)

What is Supervised Fine-Tuning (SFT)?



Source: Argilla, "Collecting RLHF data" (docs.argilla.io); left panel from Ouyang et al., 2022.

Supervised Fine-Tuning For Gemini LLM



Supervised Fine Tuning for Gemini LLM

Source: E. Huizenga & M. Hu, "When to use supervised fine-tuning for Gemini," Google Cloud Blog, 2024.

► Instructional datasets:

- **OpenAI:** InstructGPT, roughly 13k labeler-written demonstrations (never released publicly)
 - **Stanford Alpaca:** 52k instructions generated with OpenAI's text-davinci-003 using the Self-Instruct recipe
 - **Others:** databricks-dolly-15k, ShareGPT, OASST1, UltraChat
- Note where each one came from: Dolly and OASST1 are human-written, Alpaca and UltraChat are model-generated, ShareGPT is scraped conversations.
- **Note:** Quality of supervision greatly affects model behavior.

- ▶ General instruction data teaches broad helpfulness; **domain-specific SFT** adapts a model to a vertical using labeled in-domain data.
- ▶ **Finance datasets:**
 - **FiQA** (WWW'18 challenge): aspect-based financial sentiment and opinion-based question answering.
 - **Financial PhraseBank** (Malo et al., 2014): 4,845 financial-news sentences labeled positive / negative / neutral from an investor's point of view by annotators with a finance background.
 - **SEC filings (10-K)**: long regulatory filings used to tune models that read earnings reports and risk statements.

- ▶ **Fine-tune an existing model. FinBERT** (Araci, 2019) is this recipe exactly: BERT further trained on a Reuters financial corpus, then fine-tuned on Financial PhraseBank. It reaches 0.86 accuracy where a same-size general model reaches 0.83.
- ▶ **Pre-train from scratch on a mixed corpus. BloombergGPT** used 363B financial and 345B general tokens. Far more expensive, and worth it only with proprietary data at that scale.
- ▶ For almost everyone the first route is the right one, and the same recipe applies to law, medicine, and code: a modest, high-quality in-domain set often beats a much larger generic one.

- ▶ Cannot capture nuanced human preferences or values.
- ▶ May reinforce existing biases or hallucinations present in the data.
- ▶ Risk of overfitting, especially on synthetic or noisy datasets.
- ▶ Often leads to safe but bland and generic responses.

RLHF — Reinforcement Learning with Human Feedback

- ▶ SFT doesn't teach the model what humans prefer.
- ▶ RLHF introduces reward learning and policy optimization.
- ▶ Inspired by InstructGPT (Ouyang et al., 2022).

1. **Supervised Fine-Tuning (SFT):** Train the base model on instruction-response pairs.
2. **Reward Model (RM) Training:** Learn a reward function from human preference rankings.
3. **RL Optimization (e.g., PPO):** Optimize the language model to maximize the learned reward.

Step 1

Collect demonstration data, and train a supervised policy.

A prompt is sampled from our prompt dataset.



A labeler demonstrates the desired output behavior.



This data is used to fine-tune GPT-3 with supervised learning.



Step 2

Collect comparison data, and train a reward model.

A prompt and several model outputs are sampled.



A labeler ranks the outputs from best to worst.



This data is used to train our reward model.



Step 3

Optimize a policy against the reward model using reinforcement learning.

A new prompt is sampled from the dataset.



The policy generates an output.



Once upon a time...

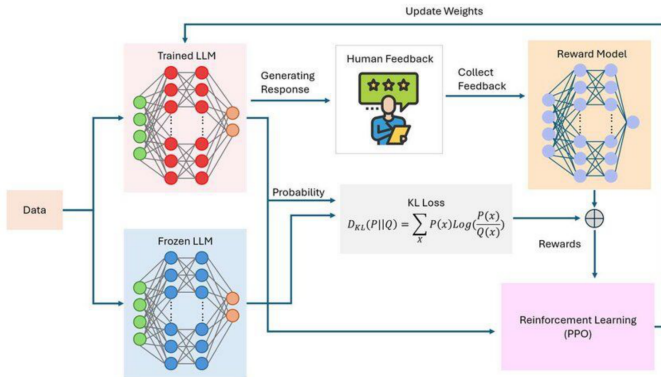
The reward model calculates a reward for the output.



The reward is used to update the policy using PPO.



Ouyang et al., "Training language models to follow instructions with human feedback," NeurIPS 2022, Fig. 2.



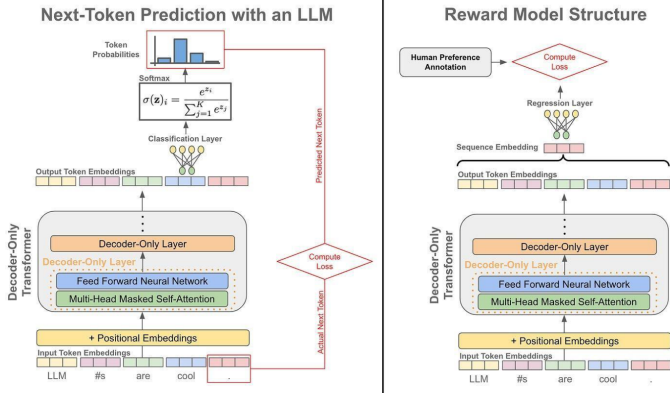
Source: Wu, You & Du, "AI generations: from AI 1.0 to AI 4.0," *Frontiers in Artificial Intelligence* 8:1585629, 2025, Fig. 7.

- ▶ **Given:** A prompt x and two responses to it. The annotator marks one as preferred, giving a winner y_w and a loser y_l .
- ▶ **Architecture:** Start from the SFT model and replace its next-token output layer with a scalar head, so $r_\phi(x, y)$ returns one score for a prompt-response pair.
- ▶ **Pairwise loss** (the Bradley–Terry preference model):

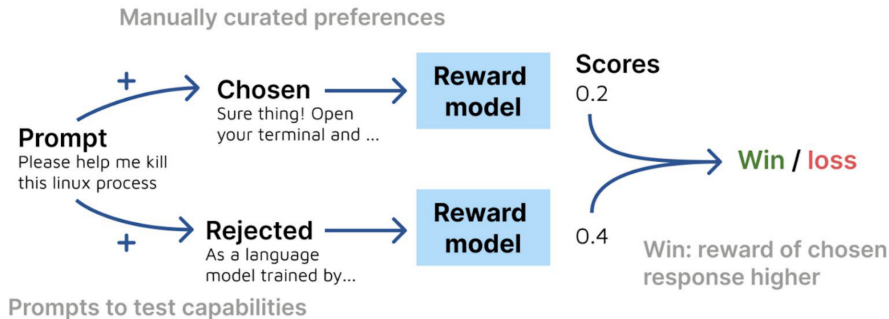
$$L_{\text{RM}} = -\log \sigma \left(r_\phi(x, y_w) - r_\phi(x, y_l) \right)$$

where σ is the sigmoid function.

- ▶ The loss constrains only the *gap* between the two scores, so the absolute reward scale is arbitrary.
- ▶ The RM acts as a proxy for human judgment in downstream RL optimization.



Source: Cameron R. Wolfe, "The Story of RLHF," Deep (Learning) Focus, 2023.



Source: Lambert et al., "RewardBench: Evaluating Reward Models for Language Modeling," 2024, Fig. 1.

Text generation is cast as a sequential decision problem, one token at a time:

- ▶ **Policy** π_θ : the language model itself; $\pi_\theta(a_t | s_t)$ is its next-token distribution.
- ▶ **State** s_t : the prompt plus every token generated so far.
- ▶ **Action** a_t : the next token, drawn from the vocabulary.
- ▶ **Rollout**: one complete sampled response to a prompt.
- ▶ **Reward**: the RM scores the *finished* response, so one number has to be credited back across every token that produced it.
- ▶ **Advantage** $\hat{A}_t$: how much better action a_t turned out than expected at s_t . A separate value model, trained alongside the policy, supplies that expectation.

- ▶ **Objective:** Maximize the reward given by the reward model (RM).
- ▶ **Reward with KL penalty:** the RM score is shaped so the policy cannot drift far from the SFT model:

$$R(x, y) = r_{\phi}(x, y) - \beta \log \frac{\pi_{\theta}(y | x)}{\pi_{\text{SFT}}(y | x)}$$

- ▶ **Clipped surrogate objective**, which PPO *maximises*:

$$L_{\text{PPO}} = \hat{\mathbb{E}}_t \left[\min \left(\rho_t(\theta) \hat{A}_t, \text{clip}(\rho_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right]$$

where $\rho_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\text{old}}(a_t | s_t)}$ is the probability ratio (not a reward) and $\hat{A}_t$ is the advantage estimate.

- ▶ Clipping caps how far a single update can move the policy, which is what keeps the optimization stable.

- ✓ Stable optimization: the clip bounds every policy update.
- ✓ Learns from the model's own samples, so it can improve on behaviour never demonstrated in the SFT data.
- × Expensive: each step needs fresh rollouts, and four networks are in play at once, the policy, a frozen SFT copy for the KL term, the reward model, and the value model.
- × Sensitive to reward model errors: the policy optimizes the RM, not the humans behind it.
- × Many interacting hyperparameters (β , ϵ , rollout size, value-loss weight).

- ▶ **Key idea:** Optimize the policy directly on preference pairs, with no reward model and no RL rollouts.
- ▶ **Reference:** Rafailov et al., 2023.

DPO loss, on the same preference triples (x, y_w, y_l) used to train an RM:

$$L_{\text{DPO}} = -\log \frac{\exp\left(\beta \log \frac{\pi_{\theta}(y_w|x)}{\pi_{\text{ref}}(y_w|x)}\right)}{\exp\left(\beta \log \frac{\pi_{\theta}(y_w|x)}{\pi_{\text{ref}}(y_w|x)}\right) + \exp\left(\beta \log \frac{\pi_{\theta}(y_l|x)}{\pi_{\text{ref}}(y_l|x)}\right)}$$

where y_w is the preferred response, y_l the dispreferred one, π_{θ} the policy being trained, π_{ref} the frozen reference (SFT) policy, and β controls how far π_{θ} is allowed to move away from π_{ref} .

- ▶ Since $\frac{e^a}{e^a + e^b} = \sigma(a - b)$, the loss above is the same expression written with a sigmoid:

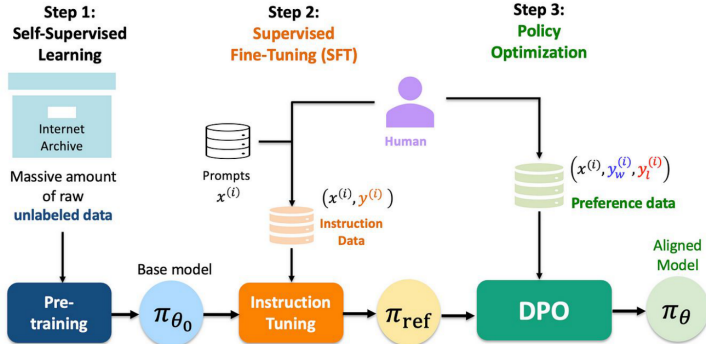
$$L_{\text{DPO}} = -\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \beta \log \frac{\pi_{\theta}(y_l | x)}{\pi_{\text{ref}}(y_l | x)} \right)$$

- ▶ Compare it with the reward-model loss $-\log \sigma(r_{\phi}(x, y_w) - r_{\phi}(x, y_l))$: the two are identical once we read $\hat{r}_{\theta}(x, y) = \beta \log \frac{\pi_{\theta}(y|x)}{\pi_{\text{ref}}(y|x)}$ as an *implicit* reward.
- ▶ The policy is therefore its own reward model, which is what the paper's title refers to. Fitting the preferences fits the policy in one step.

Benefits:

- ▶ Simpler and more stable than PPO: a plain supervised loss on a fixed dataset.
- ▶ Two networks instead of four, and no sampling loop during training.

Direct Preference Optimization (DPO)



Source: L. M. Po, "Direct Preference Optimization (DPO) of LLMs: A Paradigm Shift," *Medium*, 2024.

LoRA and Quantized LoRA

- ▶ **Key Idea:** Freeze the pre-trained weights and learn a low-rank update instead.
- ▶ For a weight matrix $W \in \mathbb{R}^{d \times k}$:

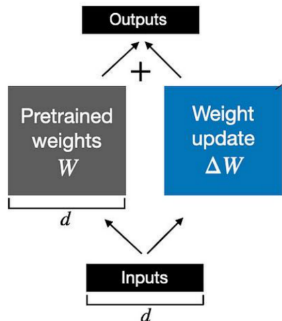
$$W' = W + \frac{\alpha}{r} \Delta W, \quad \Delta W = BA$$

with $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$, and $\text{rank } r \ll \min(d, k)$.

- ▶ Only A and B are trained: $r(d + k)$ parameters in place of dk .
- ▶ A is initialised from a Gaussian and B to zero, so $\Delta W = 0$ at the first step and training starts from exactly the pre-trained model.
- ▶ The scale α/r keeps the size of the update roughly independent of r , so changing the rank does not force a learning-rate retune.

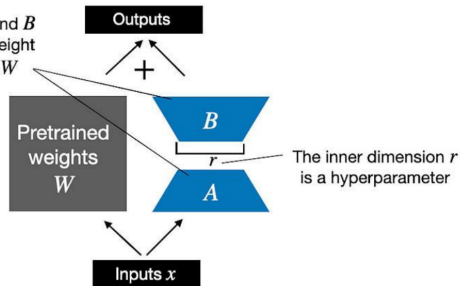
What is LoRA?

Weight update in **regular finetuning**



LoRA matrices A and B approximate the weight update matrix ΔW

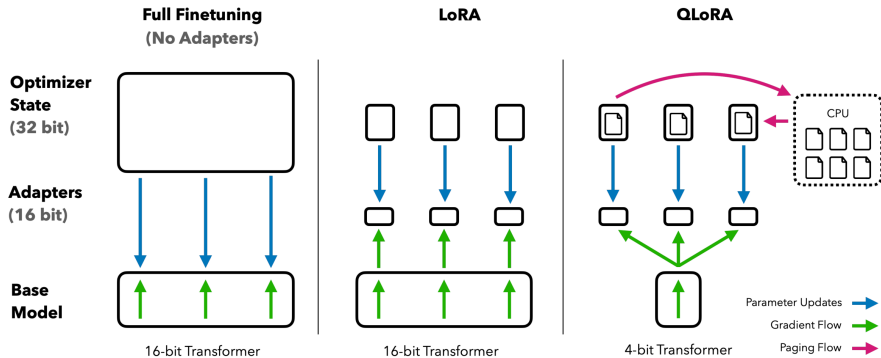
Weight update in **LoRA**



Sebastian Raschka, "Practical Tips for Finetuning LLMs Using LoRA (Low-Rank Adaptation)," *Ahead of AI*, 2023.

- ▶ **Very lightweight:** Well under 1% of parameters are trainable, and the fraction shrinks as the base model grows: about 0.2% for RoBERTa-large, and as little as 0.01% for GPT-3 175B, a $10,000\times$ cut (Hu et al., 2021).
- ▶ **Hardware friendly:** Enables fine-tuning on consumer GPUs and even laptops.
- ▶ **No inference overhead:** BA can be folded back into W once training ends, so a deployed LoRA model runs at exactly the speed of the base model.
- ▶ **Modular:** Supports plug-and-play adapters for different tasks or users.
- ▶ **Personalization:** Allows efficient user- or domain-specific adaptation.

- ▶ **Key Idea:** Fine-tune large language models in 4-bit precision using LoRA adapters.
- ▶ **Scalability:** Enables fine-tuning of 65B parameter models on a single 48GB GPU.
- ▶ **Efficiency:** Matches full 16-bit fine-tuning on the benchmarks tested; the authors note that parity at 33B and 65B was not established (Dettmers et al., 2023).
- ▶ **Techniques:**
 - 4-bit NormalFloat (NF4), a storage type matched to normally distributed weights
 - Double quantization: quantize the quantization constants too
 - Paged optimizers, to absorb memory spikes
- ▶ Weights are *stored* in 4 bits but dequantized to BFloat16 for each matrix multiply, so compute precision is unchanged.



Dettmers, Pagnoni, Holtzman & Zettlemoyer, "QLoRA: Efficient Finetuning of Quantized LLMs," NeurIPS 2023, Fig. 1.

► Instruction tuning for resource-constrained settings:

- Enables fine-tuning large models on modest hardware (e.g., consumer GPUs, laptops).
- Used for domain adaptation, personalization, and rapid prototyping.

► Models tuned this way:

- **Guanaco** (Dettmers et al., 2023): a QLoRA fine-tune of LLaMA on OpenAssistant data; the 65B model trains in 24 hours on one 48GB GPU.
- Base models such as **LLaMA** and **Mistral** are the *targets* of LoRA tuning, not products of it.
- The original **Alpaca** and **Vicuna** releases were full fine-tunes on $8\times A100$; community LoRA reproductions came later.

► Model merging and compositionality:

- Merge multiple LoRA adapters for multi-domain or multi-task capabilities.
- Compose adapters for new tasks without retraining the base model.

- ▶ **Data Quality:** RLHF relies heavily on high-quality, diverse preference data or human feedback; poor data leads to suboptimal or biased models.
- ▶ **Scalability:** Collecting human feedback at scale is expensive and time-consuming.
- ▶ **Alignment Gaps:** RLHF may not perfectly capture complex, nuanced human values or intent.
- ▶ **Overoptimization:** Models may “game” the reward model, leading to reward hacking or unnatural outputs.
- ▶ **Generalization:** Fine-tuning and RLHF can cause models to overfit to specific tasks or feedback distributions, reducing robustness.

- ▶ **Catastrophic Forgetting:** Risk of losing previously learned capabilities during aggressive fine-tuning.
- ▶ **Computational Cost:** Both fine-tuning and RLHF are resource-intensive, especially for large models.
- ▶ **Ethical Concerns:** Human preferences used for training may encode societal biases, under-representation, or other ethical issues.
- ▶ **Transparency:** RLHF pipelines can be hard to interpret and debug.

- ▶ **RLAIF (RL from AI Feedback):** Replace the human preference labeler with an off-the-shelf LLM, so preference data can be generated at scale (Bai et al., 2022).
- ▶ **Constitutional AI:** A written list of principles drives two stages: the model first critiques and revises its own responses, then AI-labeled preferences supply the RLHF reward signal (Bai et al., 2022).
- ▶ **Open RLHF Datasets:** Promote better community sharing and reproducibility with open preference datasets.
- ▶ **Multi-modal Alignment:** Extend alignment techniques to vision, audio, and code generation tasks.
- ▶ **Reward Hacking Defense:** Build reward models that hold up under heavy optimization, since gold-standard quality starts to fall once the proxy reward is pushed too far (Gao et al., 2023).

- ▶ Fine-tuning enables specialization and alignment of LLMs.
- ▶ Supervised Fine-Tuning (SFT) is the foundational step.
- ▶ RLHF incorporates human preference alignment using PPO.
- ▶ DPO streamlines the process with a direct preference loss.
- ▶ LoRA and QLoRA make adaptation efficient and accessible.
- ▶ Which method to reach for is largely a budget question: SFT to add a capability, preference tuning to shape behaviour, LoRA or QLoRA when memory is the binding constraint.

1. Ouyang, L., Wu, J., Jiang, X., et al. (2022). Training language models to follow instructions with human feedback (InstructGPT). <https://arxiv.org/abs/2203.02155>
2. Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal Policy Optimization Algorithms. <https://arxiv.org/abs/1707.06347>
3. Rafailov, R., Sharma, A., Mitchell, E., et al. (2023). Direct Preference Optimization: Your Language Model is Secretly a Reward Model. <https://arxiv.org/abs/2305.18290>
4. Bai, Y., Jones, A., Ndousse, K., et al. (2022). Training a Helpful and Harmless Assistant with Reinforcement Learning from Human Feedback. <https://arxiv.org/abs/2204.05862>
5. Bai, Y., Kadavath, S., Kundu, S., et al. (2022). Constitutional AI: Harmlessness from AI Feedback. <https://arxiv.org/abs/2212.08073>
6. Gao, L., Schulman, J., & Hilton, J. (2023). Scaling Laws for Reward Model Overoptimization. ICML 2023. <https://arxiv.org/abs/2210.10760>
7. Lambert, N., Castriato, L., von Werra, L., & Havrilla, A. (2022). Illustrating Reinforcement Learning from Human Feedback (RLHF). Hugging Face Blog. <https://huggingface.co/blog/rlhf>
8. Lambert, N., Pyatkin, V., Morrison, J., et al. (2024). RewardBench: Evaluating Reward Models for Language Modeling. <https://arxiv.org/abs/2403.13787>

9. Houslsby, N., Giurgiu, A., Jastrzebski, S., et al. (2019). Parameter-Efficient Transfer Learning for NLP. ICML 2019. <https://arxiv.org/abs/1902.00751>
10. Li, X. L., & Liang, P. (2021). Prefix-Tuning: Optimizing Continuous Prompts for Generation. ACL-IJCNLP 2021. <https://arxiv.org/abs/2101.00190>
11. Lester, B., Al-Rfou, R., & Constant, N. (2021). The Power of Scale for Parameter-Efficient Prompt Tuning. EMNLP 2021. <https://arxiv.org/abs/2104.08691>
12. Hu, E. J., Shen, Y., Wallis, P., et al. (2021). LoRA: Low-Rank Adaptation of Large Language Models. <https://arxiv.org/abs/2106.09685>
13. Dettmers, T., Pagnoni, A., Holtzman, A., & Zettlemoyer, L. (2023). QLoRA: Efficient Finetuning of Quantized LLMs. <https://arxiv.org/abs/2305.14314>
14. Taori, R., Gulrajani, I., Zhang, T., et al. (2023). Stanford Alpaca: An Instruction-following LLaMA Model. Stanford CRFM. <https://crfm.stanford.edu/2023/03/13/alpaca.html>
15. Araci, D. (2019). FinBERT: Financial Sentiment Analysis with Pre-trained Language Models. <https://arxiv.org/abs/1908.10063>
16. Malo, P., Sinha, A., Korhonen, P., Wallenius, J., & Takala, P. (2014). Good debt or bad debt: Detecting semantic orientations in economic texts. *JASIST* 65(4), 782–796.
17. Wu, S., Irsoy, O., Lu, S., et al. (2023). BloombergGPT: A Large Language Model for Finance. <https://arxiv.org/abs/2303.17564>